

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training : (N_train, 128, 9)
Validation : (N_val, 128, 9)
Testing : (N_test, 128, 9)
```

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Actual Preprocessing Output

The preprocessing performed in the notebook produced the following input tensor shapes:

Training : (2100, 128, 9)
Validation : (450, 128, 9)
Testing : (450, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The class distribution was balanced, with 350 samples per class in the training set, 75 samples per class in the validation set, and 75 samples per class in the test set.

Inference: The preprocessing successfully produced balanced training, validation, and test sets while preserving the 128-step temporal structure and 9 sensor features of each observation window.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Temporal Sensor Signal Plots

The temporal sensor signals for three representative activities are shown below. Each plot contains the Body Acceleration X, Y, and Z channels over 128 time steps.

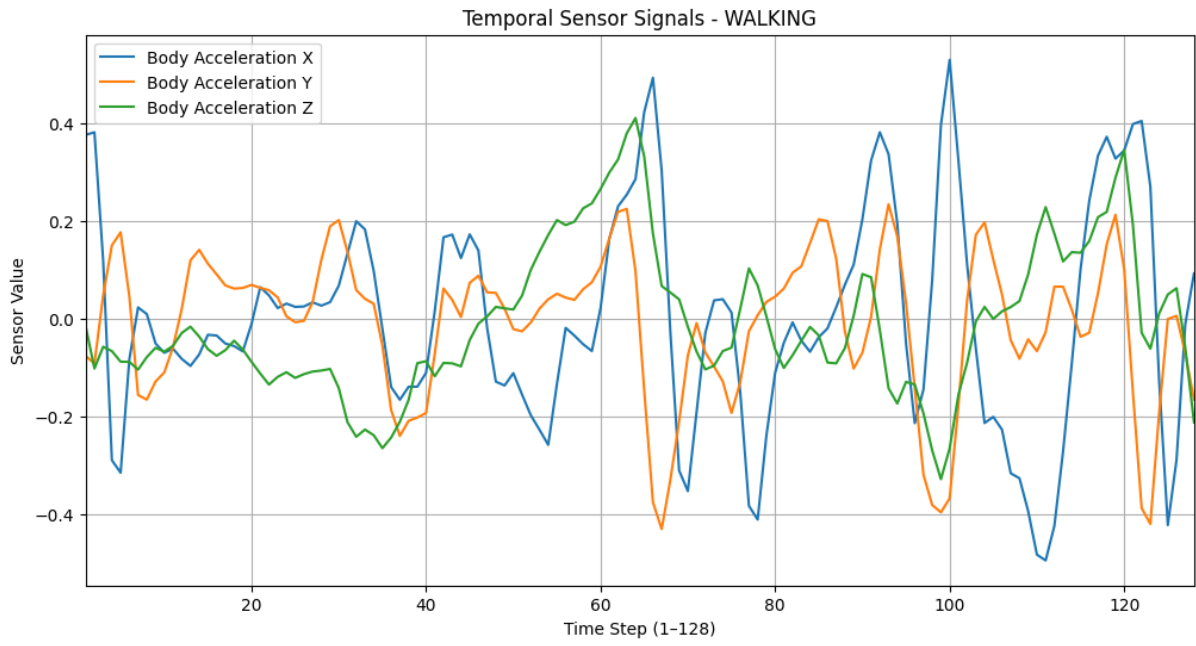


Figure 1: Temporal sensor signals for Walking

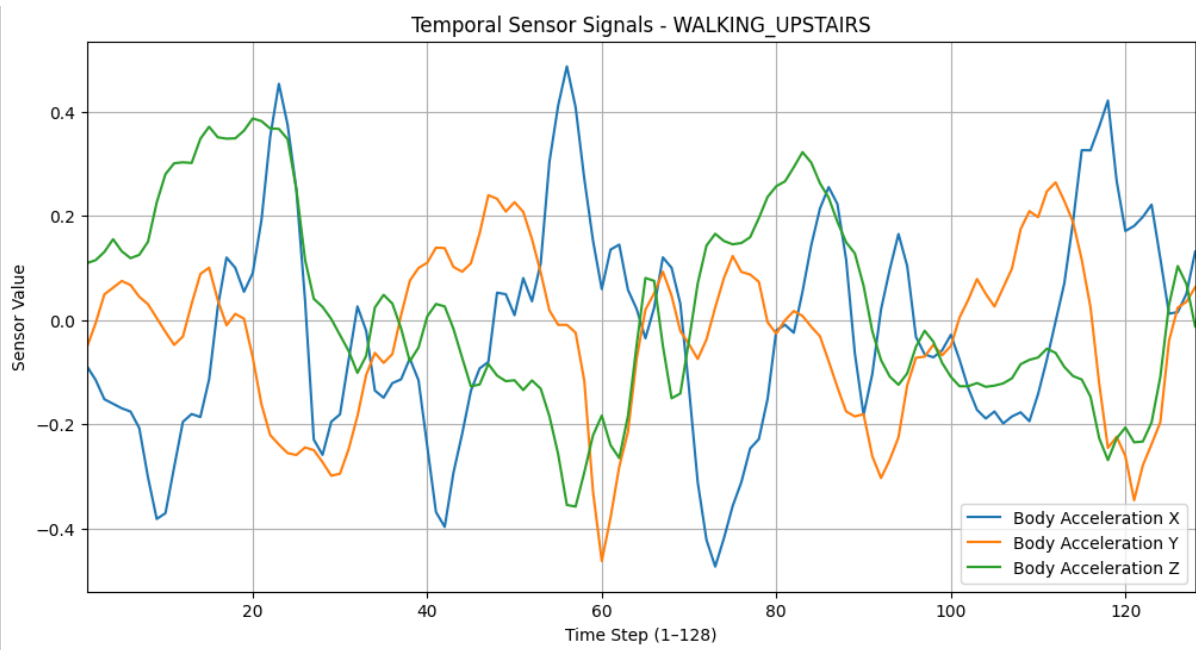


Figure 2: Temporal sensor signals for Walking Upstairs

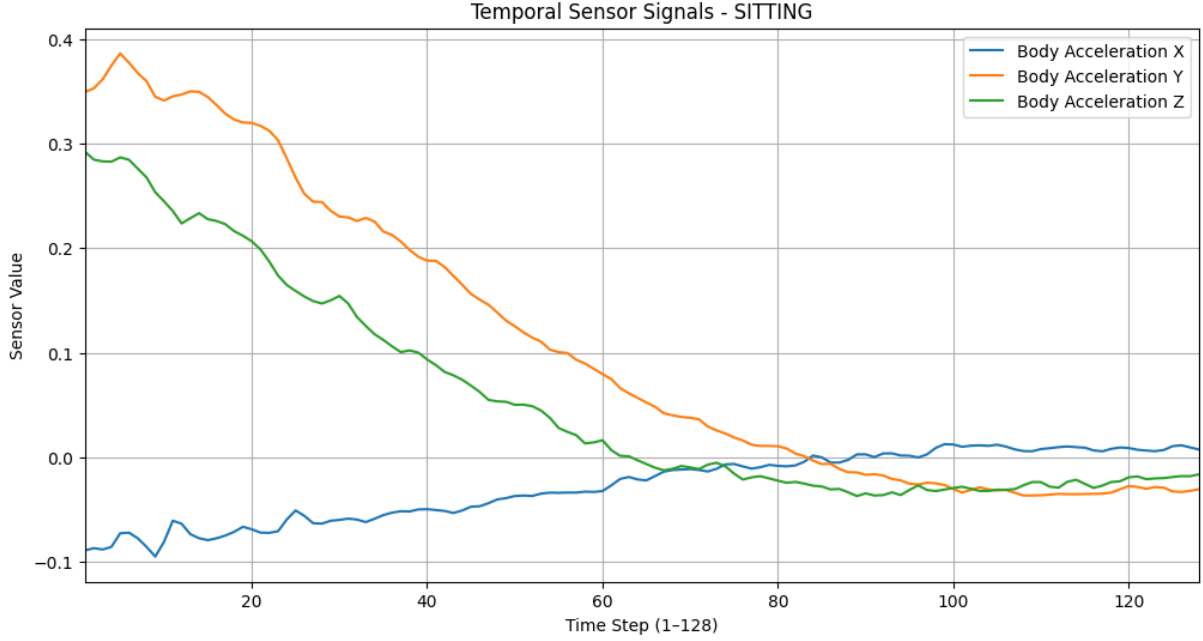


Figure 3: Temporal sensor signals for Sitting

Required Inference

1. What temporal pattern is visible?

The Walking and Walking Upstairs signals show clear fluctuations across the 128 time steps, especially in the X and Y acceleration channels. In contrast, the Sitting signals change more gradually and become relatively stable toward the later time steps. This shows that movement-based activities produce more noticeable temporal variations than Sitting.

2. Which activities exhibit similar patterns?

Walking and Walking Upstairs show some similarities because both involve continuous movement and have noticeable fluctuations in their acceleration signals. However, the timing and magnitude of these fluctuations are different. Sitting has a noticeably smoother pattern compared with the two walking activities.

3. Why is the ordering of measurements important?

The ordering is important because the activity is represented not only by individual sensor values but also by how those values change over time. Preserving the sequence allows the RNN, LSTM, and GRU models to learn the temporal patterns associated with each activity.

The three activities can be distinguished from their temporal behaviour. Walking and Walking Upstairs contain repeated changes in acceleration, whereas Sitting shows a much smoother progression. This supports the use of sequence-based models for activity recognition, since the temporal order contains useful information about the activity being performed.

7. Vanilla RNN Architecture

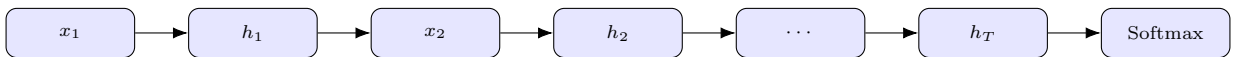
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

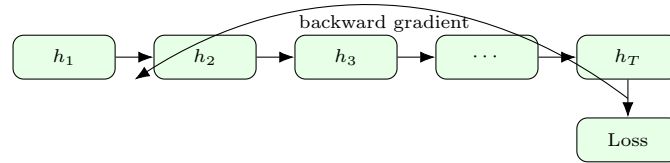
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

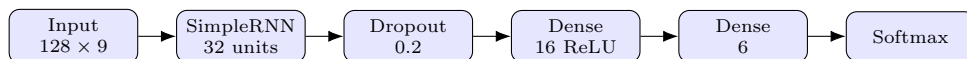
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

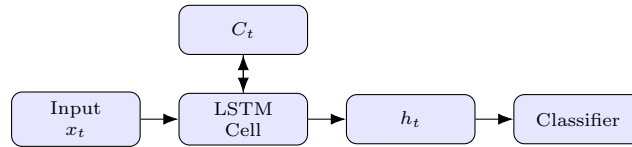
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

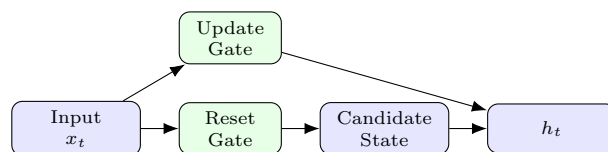
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Training and Validation Plots

RNN

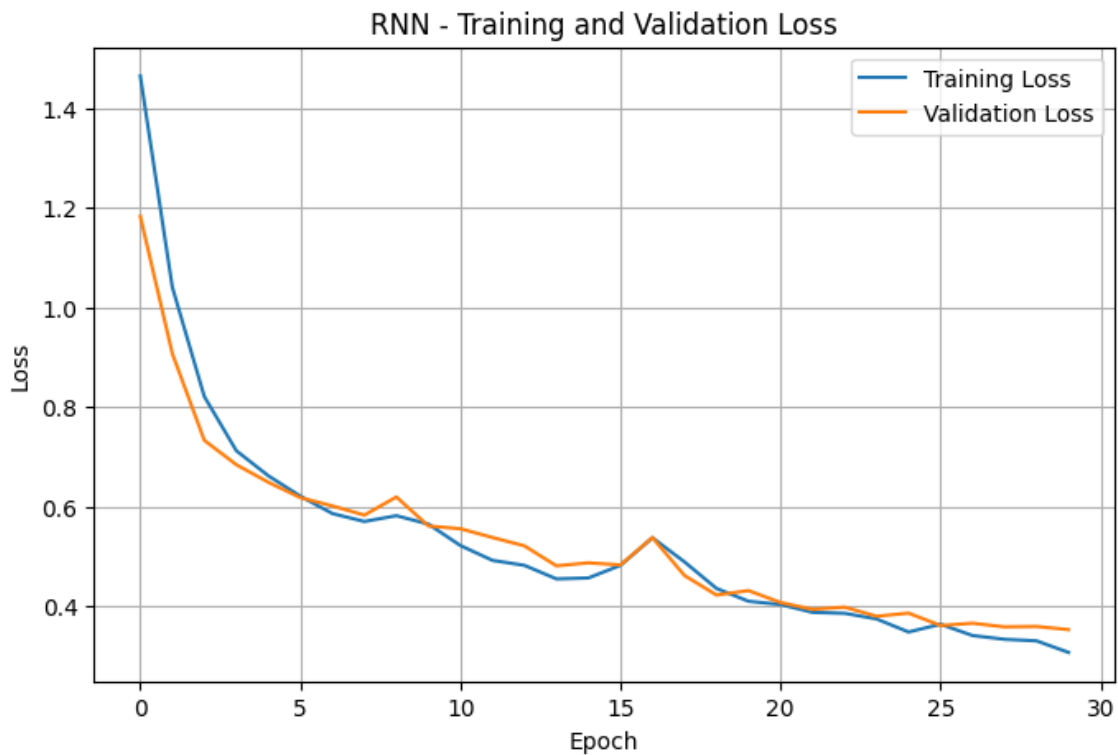


Figure 4: RNN training and validation loss

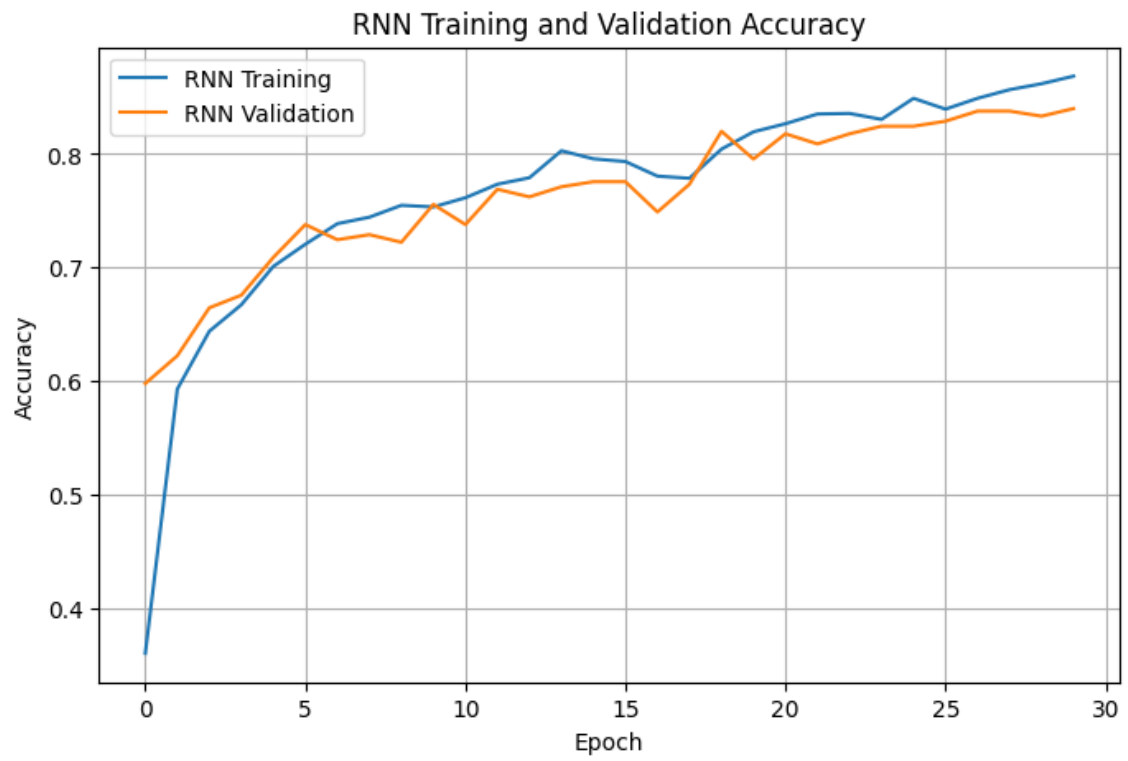


Figure 5: RNN training and validation accuracy

LSTM

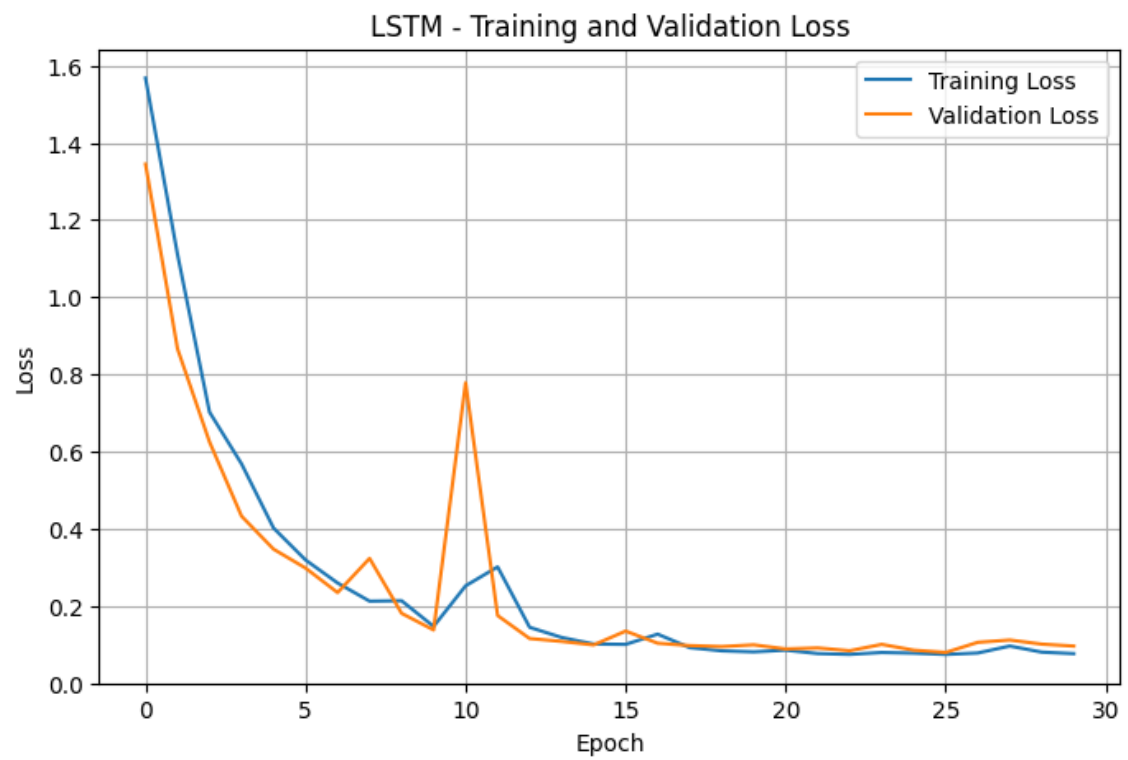


Figure 6: LSTM training and validation loss

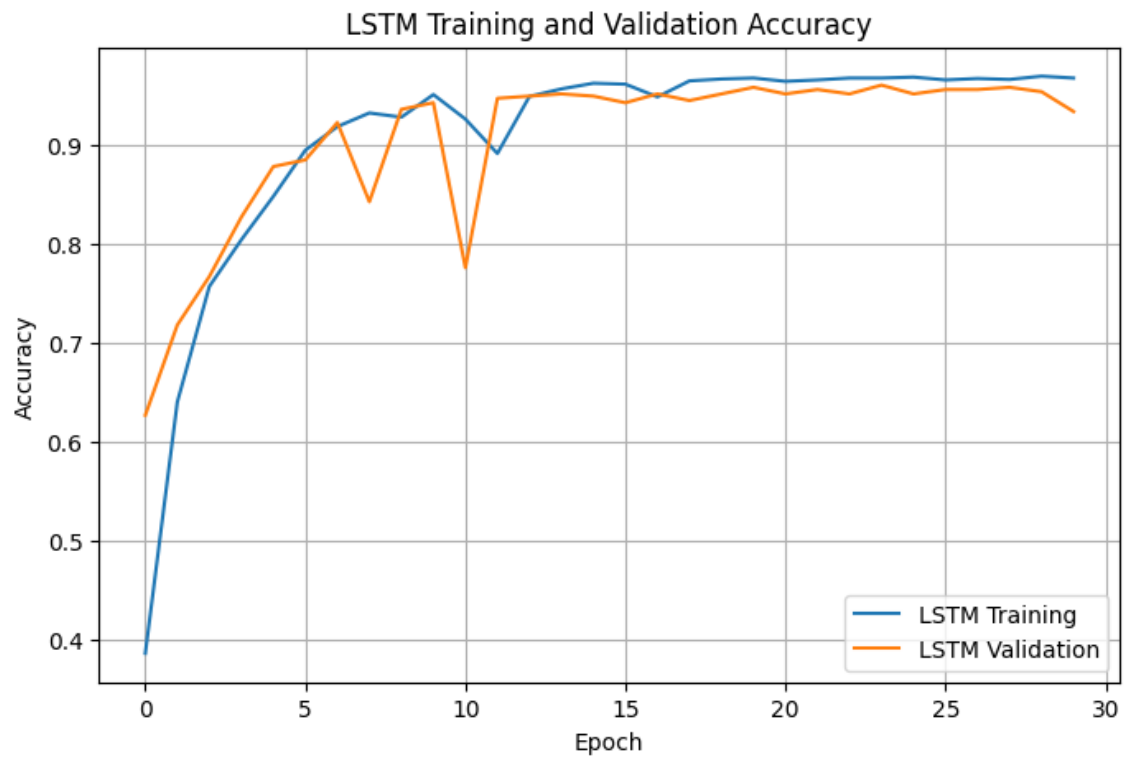


Figure 7: LSTM training and validation accuracy

GRU

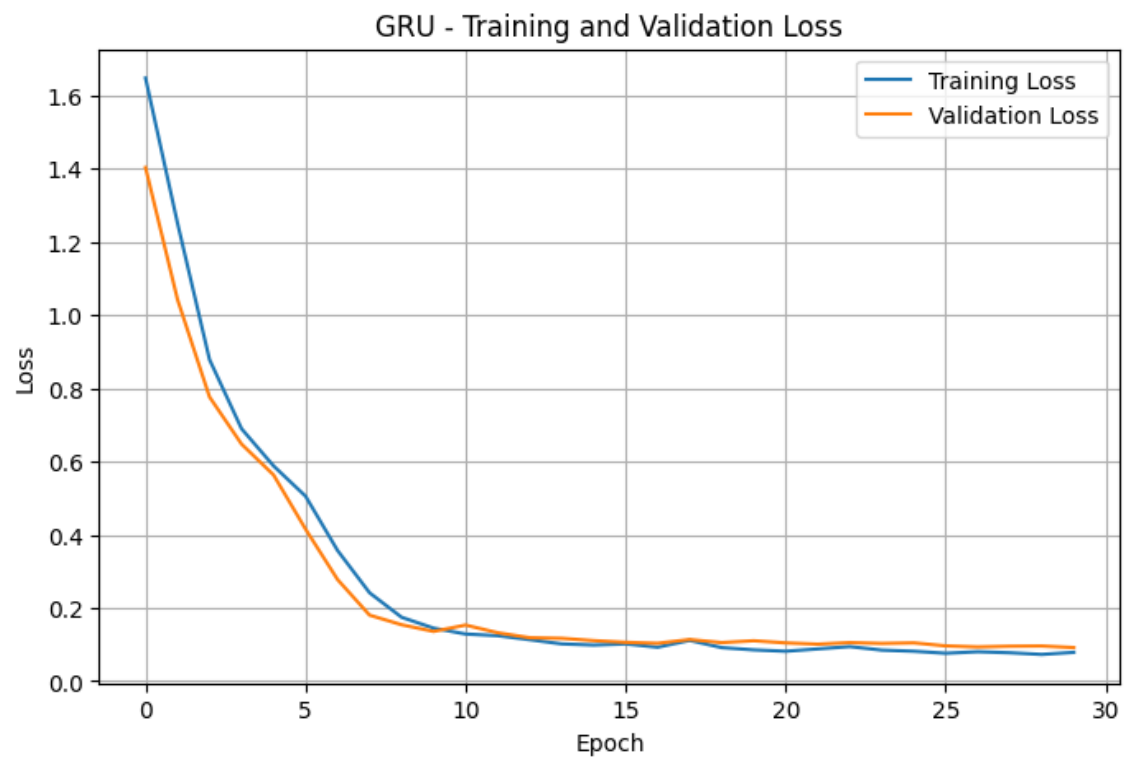


Figure 8: GRU training and validation loss

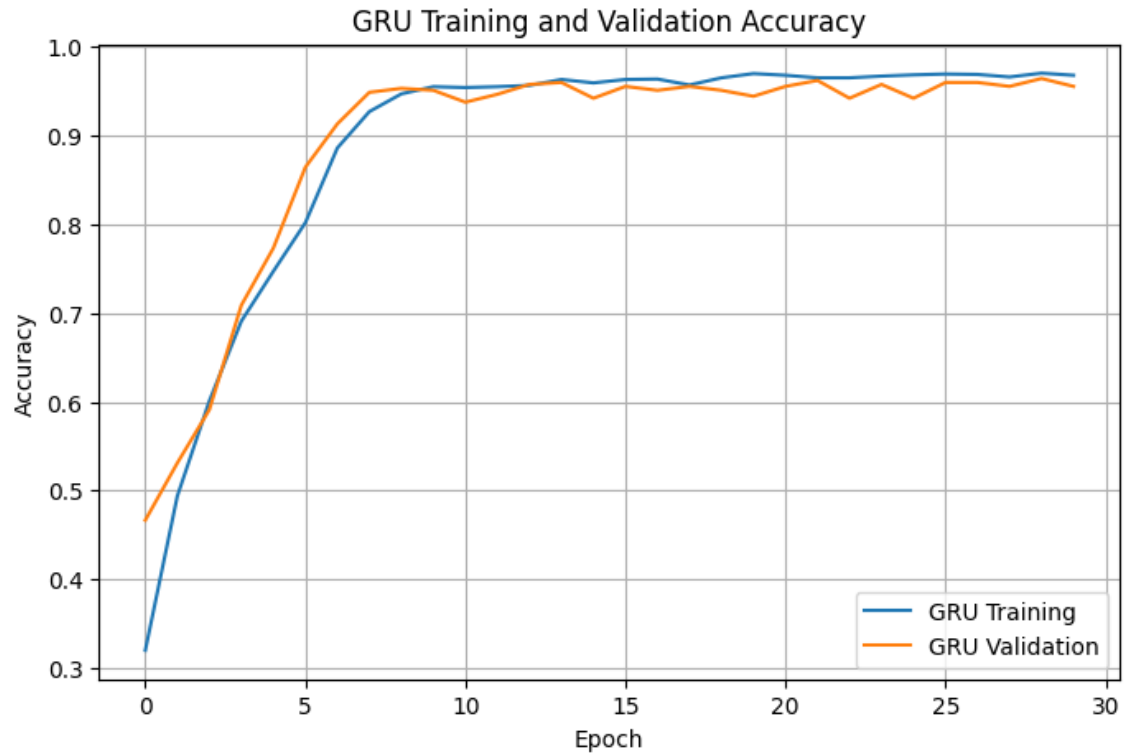


Figure 9: GRU training and validation accuracy

Inference

- **RNN:** The RNN shows a steady improvement in accuracy and a gradual decrease in loss over the epochs. The training and validation curves remain relatively close, although some fluctuations are visible during training. The model appears to converge without a large generalization gap, and there is no clear sign of persistent overfitting.
- **LSTM:** The LSTM learns quickly during the initial epochs, with both training and validation accuracy increasing substantially. There are some temporary fluctuations in the validation curves, particularly around the middle of training, but the validation performance recovers and becomes stable later. The training and validation loss also decrease to low values, indicating good convergence. There is no strong persistent overfitting visible from the curves.
- **GRU:** The GRU shows rapid improvement during the early epochs and reaches a high and stable accuracy after the initial learning phase. The training and validation accuracy curves remain close to each other for most of the later epochs. The training and validation losses also decrease and remain close, suggesting stable convergence with a relatively small generalization gap.
- **Overall observation:** All three models show convergence as the loss decreases and the accuracy increases during training. No model shows a clear, persistent overfitting pattern where training performance keeps improving while validation performance continuously deteriorates. Similarly, there is no strong evidence of underfitting in the final stages of training. The LSTM and GRU curves reach a more stable high-accuracy region than the RNN, which is consistent with their stronger performance in the later evaluation.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)			
Macro Precision (%)			
Macro Recall (%)			
Macro F1 (%)			
Parameters			
Training Time (s)			

Actual Performance Results

The three models were evaluated on the independent test set using accuracy, macro-averaged precision, macro-averaged recall and macro-averaged F1-score. The number of trainable parameters and training time were also recorded.

Metric	RNN	LSTM	GRU
Accuracy (%)	84.00	94.22	94.67
Macro Precision (%)	84.00	94.13	94.60
Macro Recall (%)	84.00	94.22	94.67
Macro F1 (%)	83.99	94.17	94.62
Parameters	1974	6006	4758
Training Time (s)	58.43	116.33	180.96

Inference: The LSTM and GRU models achieved substantially higher test performance than the RNN. The GRU obtained the highest test accuracy and macro F1-score among the three models, while the RNN had the smallest number of parameters and the shortest recorded training time. This shows the trade-off between model complexity, training cost and predictive performance.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Confusion Matrices

The confusion matrices obtained on the independent test set are shown below.

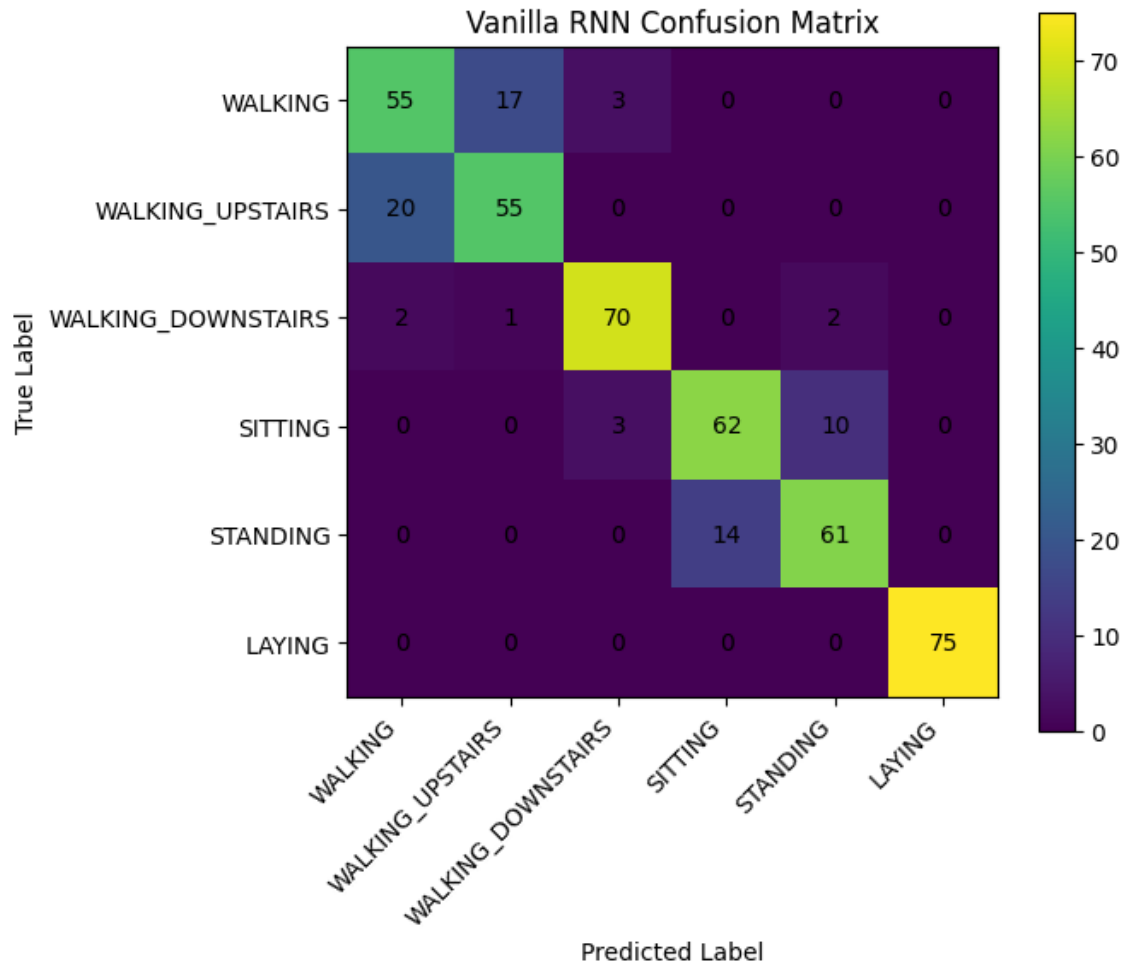


Figure 10: RNN confusion matrix on the test set

RNN: The RNN correctly recognized LAYING in all test samples, giving a recognition rate of 100%. Its largest number of misclassifications occurred for WALKING, with 20 samples being classified incorrectly. The most common confusion was between WALKING_UPSTAIRS and WALKING, followed by confusion between WALKING and WALKING_UPSTAIRS. SITTING and STANDING were also occasionally confused.

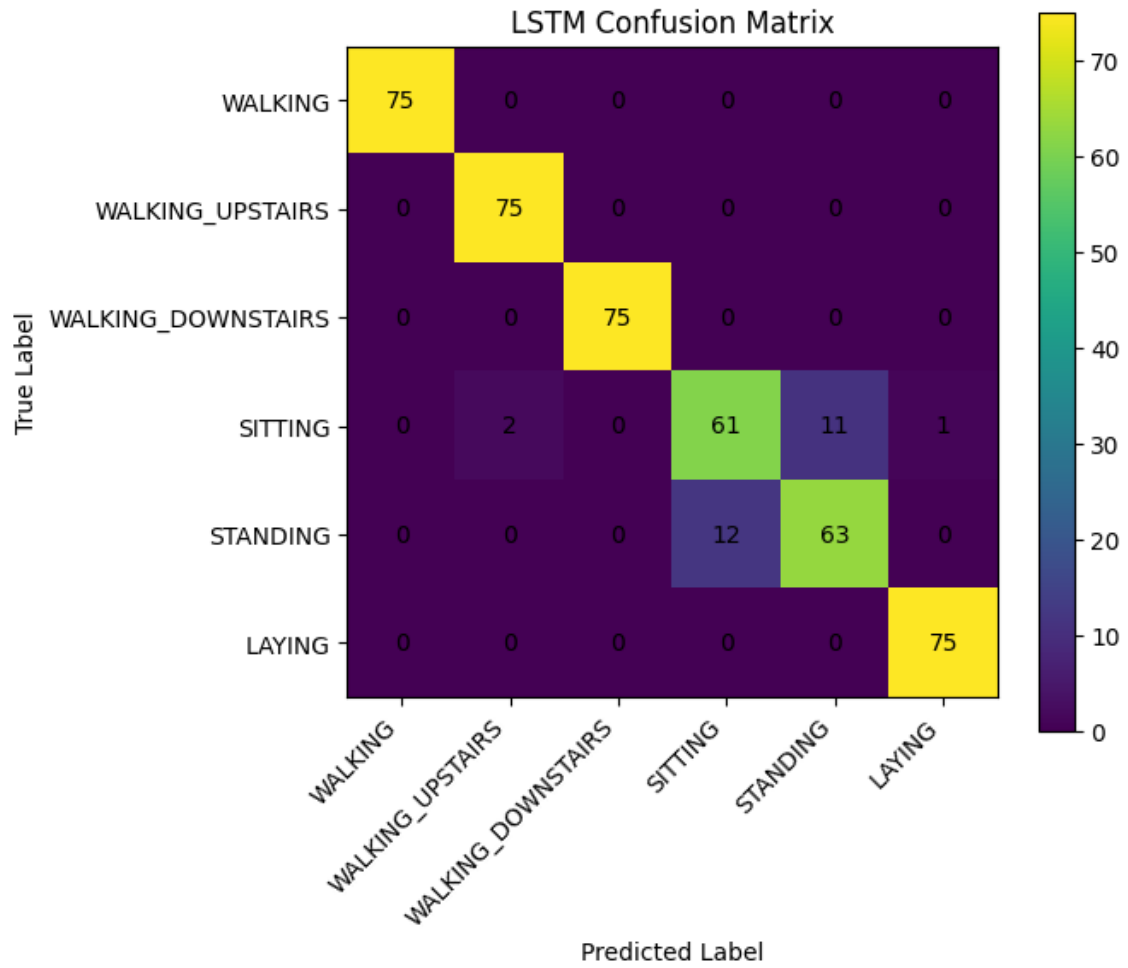


Figure 11: LSTM confusion matrix on the test set

LSTM: The LSTM achieved 100% recognition for WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS and LAYING. Most of its remaining errors were between the stationary activities SITTING and STANDING. SITTING had the largest number of misclassifications, with 14 samples classified incorrectly. The most frequent confusion was STANDING → SITTING with 12 samples.

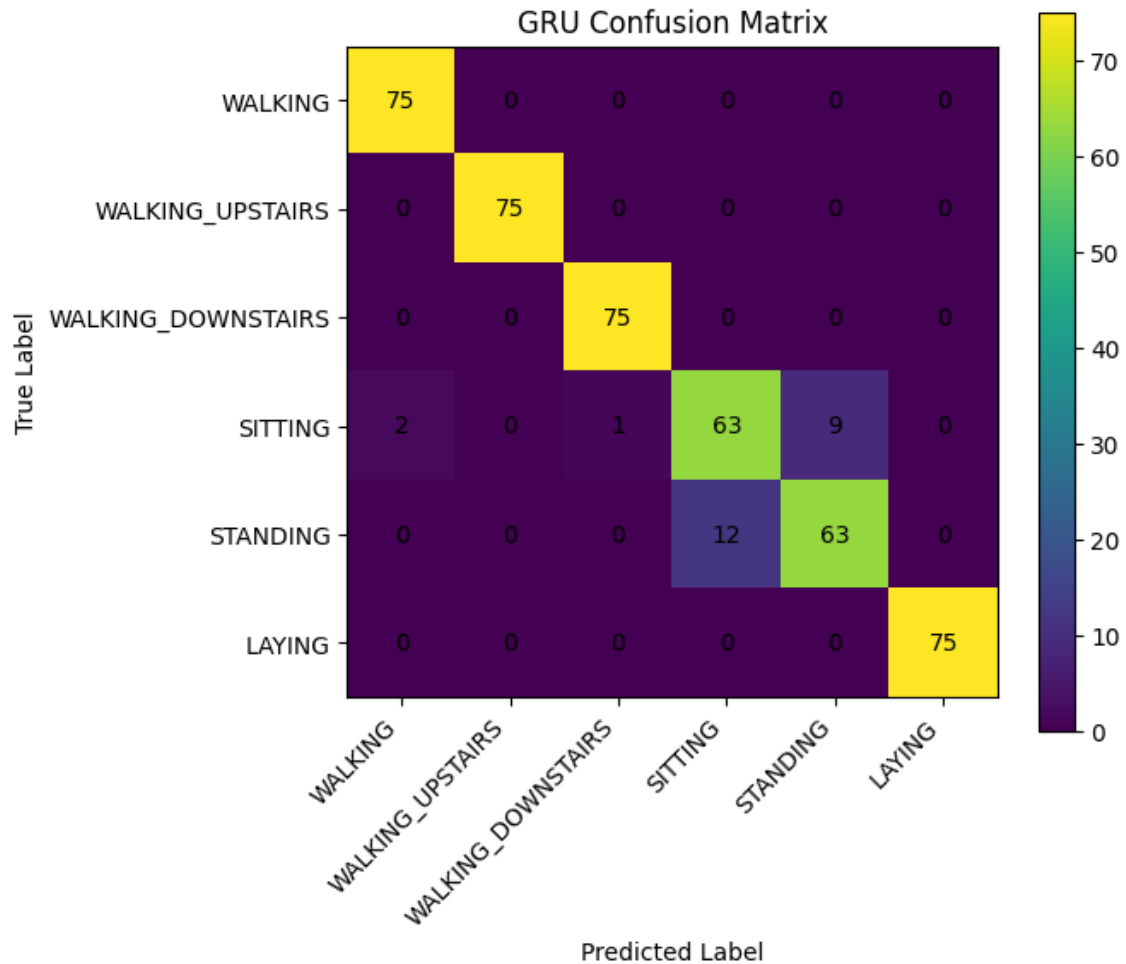


Figure 12: GRU confusion matrix on the test set

GRU: The GRU also achieved 100% recognition for the three walking activities and LAYING. Its main errors were again between SITTING and STANDING. SITTING had the largest number of misclassifications with 12 samples. The most frequent confusion was STANDING → SITTING with 12 samples, followed by SITTING → STANDING with 9 samples.

Overall inference: The confusion matrices show that the walking activities and LAYING were easier for the LSTM and GRU to distinguish, while SITTING and STANDING were more difficult because their sensor patterns are more similar. The LSTM and GRU therefore produced fewer classification errors than the RNN, especially for the movement-based activities.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters			
Training time			
Test F1-score			

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost}.$$

Completed Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	58.43	116.33	180.96
Test F1-score (%)	83.99	94.17	94.62

Plot 5: Model Performance Comparison

The test accuracy and macro F1-score of the three recurrent models are compared below.

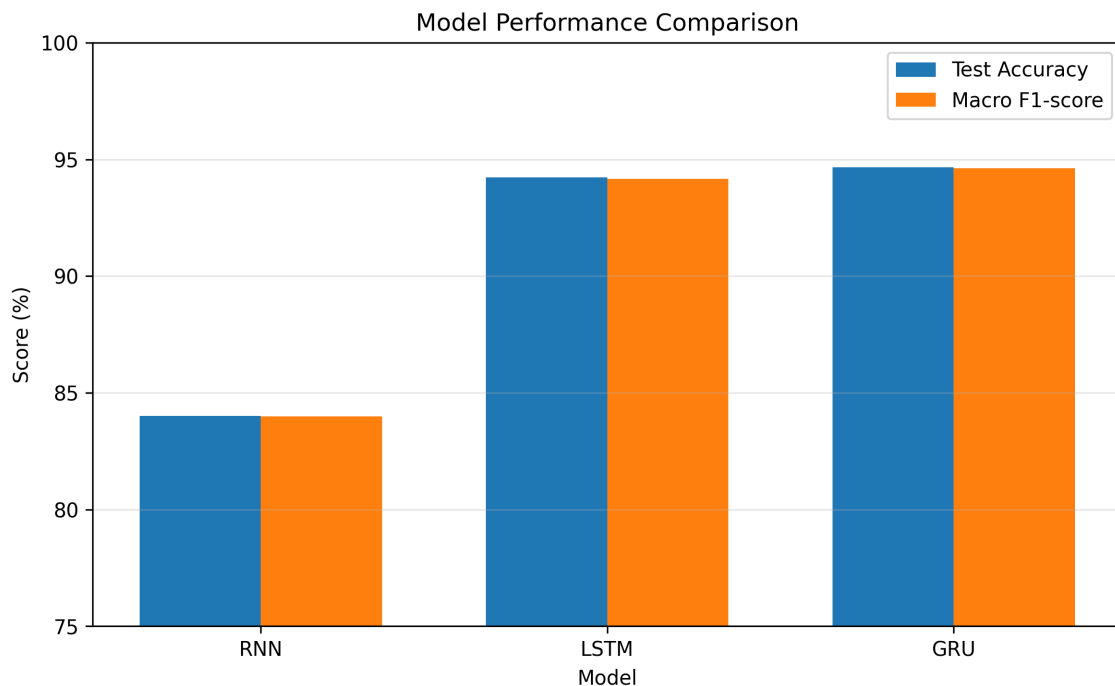


Figure 13: Comparison of test accuracy and macro F1-score for RNN, LSTM and GRU

Required Inference

The comparison shows a clear trade-off between model complexity, training cost and predictive performance. The RNN has the fewest parameters and the shortest training time, but its test accuracy and F1-score are lower than those of the LSTM and GRU.

The LSTM and GRU require more parameters and longer training times, but both achieve substantially higher test performance. In this experiment, the GRU has fewer parameters than the LSTM and achieves a slightly higher test accuracy and macro F1-score, while its measured training time is longer. Overall, the results show that the

simpler RNN is less computationally expensive, whereas the gated recurrent models provide stronger classification performance at a higher computational cost.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32			
64			
128			

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

Actual Results

The experiment was repeated using sequence lengths of 32, 64, and 128 time steps. The resulting Macro F1-scores on the test set are shown below.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	76.3615	95.3201	93.9724
64	73.2813	94.8720	94.8726
128	84.0260	95.3003	95.3165

Plot 6: Sequence Length vs. Test F1-score

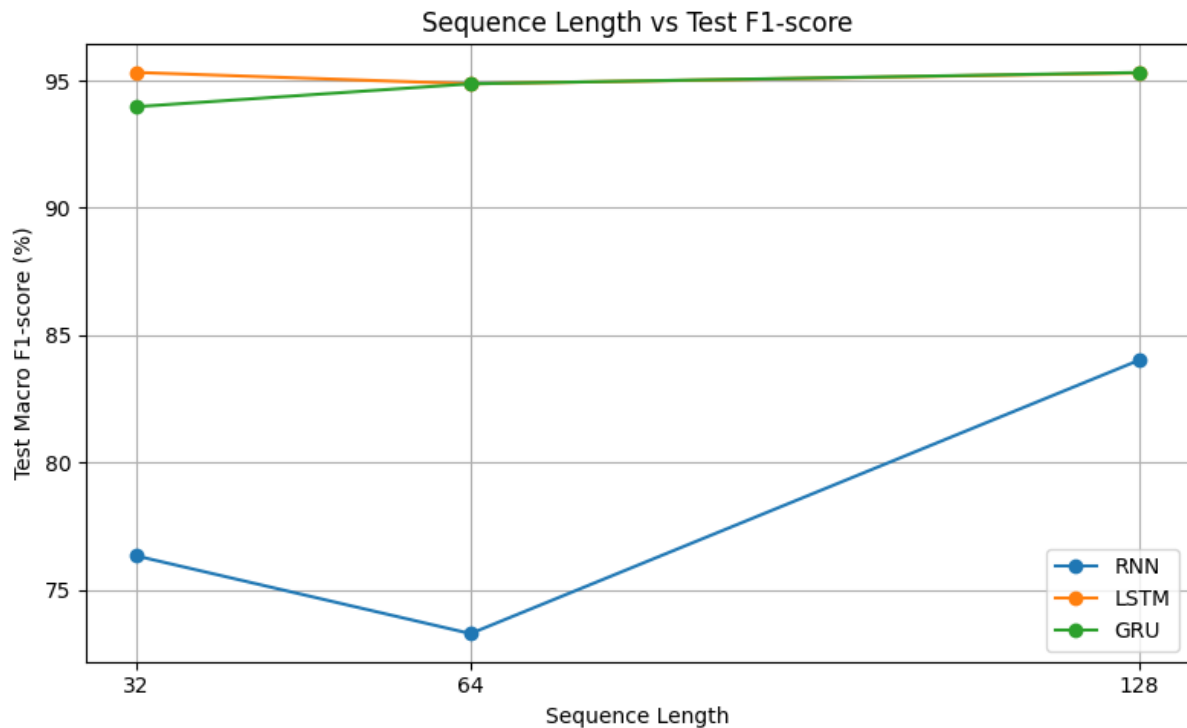


Figure 14: Effect of sequence length on test Macro F1-score for RNN, LSTM, and GRU

Inference

The effect of sequence length is different for the three models. The RNN achieves its highest F1-score at 128 time steps (84.03%), while its performance decreases at 64 time steps (73.28%).

The LSTM maintains a consistently high F1-score across all three sequence lengths, with values around 95%. Its highest value is 95.32% at a sequence length of 32.

The GRU also maintains high performance, increasing from 93.97% at 32 time steps to 95.32% at 128 time steps.

Overall, the results show that sequence length affects the models differently. The RNN benefits noticeably from the longer temporal context, whereas LSTM and GRU remain relatively stable across the tested sequence lengths.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Observation:

The pretrained MobileNetV2 model was used as a frozen feature extractor, with the classification head removed. Each video was represented using 10 uniformly sampled frames of size $224 \times 224 \times 3$.

After processing each frame through MobileNetV2 followed by Global Average Pooling, the resulting CNN feature dimension was:

$$D = 1280$$

Therefore, each video was represented as a sequence of 10 feature vectors. The resulting tensor shapes were:

Training data: (48, 10, 1280)

Testing data: (12, 10, 1280)

Thus, the recurrent network received an input sequence of shape

$$10 \times 1280$$

for each video. The CNN was kept frozen and was not trained further. This allowed the recurrent models to work with the spatial features already learned by MobileNetV2.

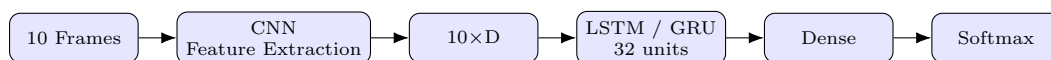
Inference:

The CNN can be thought of as looking at each frame independently and summarizing what is present in that frame. For every frame, MobileNetV2 produces a 1280-dimensional feature vector. Since 10 frames are taken from each video, these vectors form a sequence of 10 feature vectors.

The CNN mainly captures the spatial information in the frames, while the recurrent network can later use the order of these feature vectors to learn how the information changes over time. In this way, the CNN and recurrent network have different but complementary roles in understanding the video.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

Observation:

The CNN–LSTM model achieved an accuracy, precision, recall and F1-score of 1.0000 on the test set. The CNN–GRU model also achieved 1.0000 for all four metrics. Both models correctly classified all 12 test videos from the three selected action classes: Basketball, Biking and TennisSwing.

The CNN-LSTM model contains 168,163 parameters, while the CNN-GRU model contains 126,243 parameters. Thus, the GRU-based model used fewer parameters while obtaining the same measured test performance in this experiment.

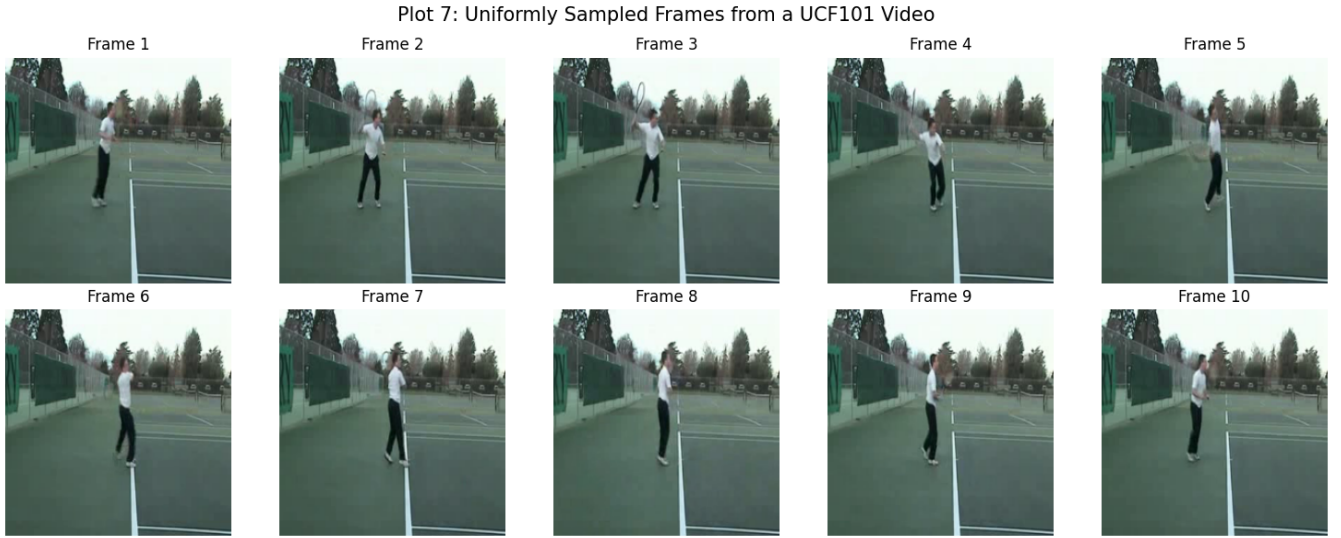


Figure 15: Ten uniformly sampled frames from a representative UCF101 video.

Observation:

The sampled frames show different points in time from the same video. The frames provide the CNN with spatial information such as the appearance of the person, objects, body position, background and other visual patterns present in each frame.

The frames are processed individually by MobileNetV2. The resulting feature vectors are then arranged in their original temporal order and supplied to the recurrent network.

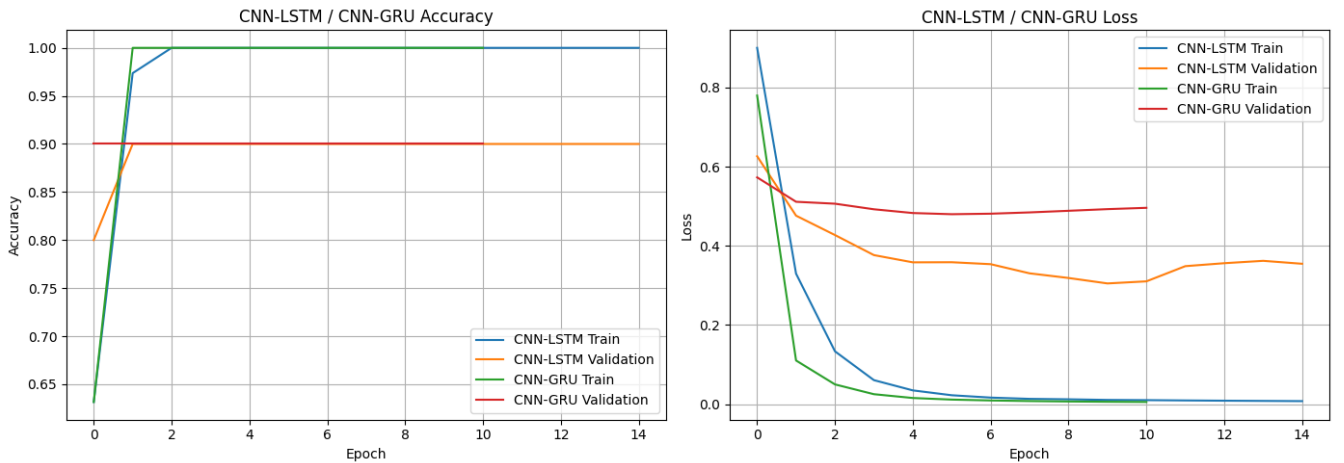


Figure 16: Training and validation accuracy and loss for CNN-LSTM and CNN-GRU models.

Observation:

From the training curves, both CNN-LSTM and CNN-GRU learned the training samples very quickly, with the training accuracy reaching 1.0. The validation accuracy remained around 0.90 during training. This difference is noticeable because the experiment uses a relatively small subset of videos.

Although the validation curves do not reach 1.0 throughout training, the final models correctly classified all 12 samples in the separate test set. Therefore, the final test results should be interpreted specifically for the selected three-class dataset rather than as a general performance measure for the complete UCF101 dataset.

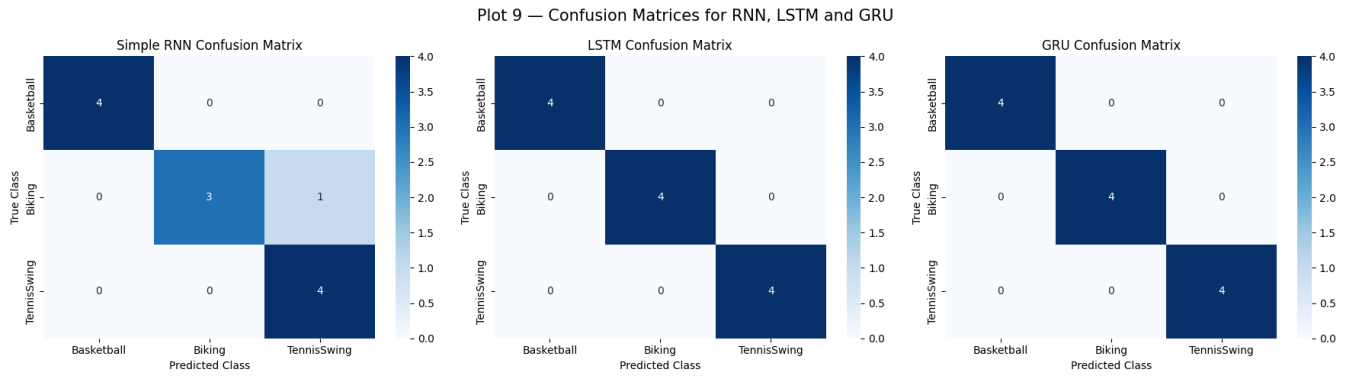


Figure 17: Confusion matrices for the recurrent video classification models.

Observation:

The confusion matrices show that the CNN–LSTM and CNN–GRU models correctly classified all the test videos. Each class has four correct predictions and there are no off-diagonal predictions.

For Basketball, Biking and TennisSwing, the recognition rate is therefore 100% on the 12-video test set. There is no class-level temporal confusion visible in the final test predictions for either model.

In comparison, the Simple RNN made one error by classifying a Biking video as TennisSwing. The LSTM and GRU models did not show this confusion on the test set, suggesting that their recurrent representations were able to capture the temporal patterns needed for this particular three-class experiment.

Inference:

The experiment shows how the two stages work together. MobileNetV2 converts each individual frame into a compact spatial representation, while the LSTM or GRU processes the sequence of these representations to use temporal information. In this experiment, both CNN–LSTM and CNN–GRU achieved perfect classification on the 12-video test set.

The CNN–GRU achieved the same measured performance as CNN–LSTM with fewer parameters (126,243 compared with 168,163). However, since the test set contains only 12 videos, these results should be considered specific to this selected subset and not as a general conclusion about performance on the complete UCF101 dataset.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Expected Output	Predicted Output
1	[1,0,3,4]	[4,3,0,1]	[4,3,0,1]
2	[1,7,7,1]	[1,7,7,1]	[1,7,7,1]
3	[6,2,5,6]	[6,5,2,6]	[6,5,2,6]
4	[9,3,9,4]	[4,9,3,9]	[4,9,3,9]
5	[7,3,8,4]	[4,8,3,7]	[4,8,3,7]

Inference:

The examples show that the model learned to produce the elements in reverse order. For example, the input sequence [6, 2, 5, 6] was transformed into [6, 5, 2, 6]. All five displayed test cases were predicted correctly, consistent with the overall test sequence accuracy of 1.0000 obtained in the experiment.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

24. Sequence-to-Sequence Evaluation

The trained Seq2Seq LSTM was evaluated using token-level and sequence-level accuracy along with the training and validation losses.

Metric	Result
Training Loss	0.0075
Validation Loss	0.0082
Test Loss	0.0080
Token Accuracy	1.0000
Sequence Accuracy	1.0000

The token accuracy was calculated as:

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}} = \frac{4000}{4000} = 1.0000$$

The sequence accuracy was calculated as:

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}} = \frac{1000}{1000} = 1.0000$$

Observation:

The model achieved a token accuracy of 1.0000 and a sequence accuracy of 1.0000 on the test set. This means that all 4000 output tokens were predicted correctly and all 1000 test sequences were completely correct. The training and validation losses were also very low, with final values of 0.0075 and 0.0082 respectively. The test loss was 0.0080.

Required Inference:

Inference:

The model learned the reversal task very well. In this test set, it did not make even a single token-level error, so every complete sequence was also correct. This is why both token accuracy and sequence accuracy reached 100%. In general, sequence accuracy is harder to achieve because one wrong token is enough to make the whole sequence incorrect, even when the model gets most of the individual tokens right.

25. Consolidated Results

The following table summarizes the classification performance obtained for the recurrent and CNN–recurrent models on the selected UCF101 test set.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	0.9167	0.9333	0.9167	0.9153	42,115
LSTM	1.0000	1.0000	1.0000	1.0000	168,163
GRU	1.0000	1.0000	1.0000	1.0000	126,243
CNN–LSTM	1.0000	1.0000	1.0000	1.0000	168,163
CNN–GRU	1.0000	1.0000	1.0000	1.0000	126,243

Sequence-to-Sequence Results

The following table summarizes the performance of the Seq2Seq LSTM on the sequence reversal task.

Metric	Result
Training Loss	0.0075
Validation Loss	0.0082
Test Loss	0.0080
Token Accuracy	1.0000
Sequence Accuracy	1.0000

Observation:

The Simple RNN achieved an accuracy of 91.67%, while the LSTM and GRU models achieved 100% accuracy on the selected test set. The CNN–LSTM and CNN–GRU models also achieved 100% accuracy. The GRU-based models used fewer parameters than their corresponding LSTM models while obtaining the same measured test performance in this experiment.

The Seq2Seq LSTM also performed successfully on the sequence reversal task, achieving 100% token accuracy and 100% sequence accuracy. All 4000 test tokens and all 1000 test sequences were predicted correctly.

These results are based on the selected three-class UCF101 subset and the 12-video test set used in this experiment. Therefore, the results describe the behaviour of the implemented models on this experimental subset rather than the performance of the models on the complete UCF101 dataset.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;

- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

- 1. What is a recurrent neural network?**
A Recurrent Neural Network (RNN) is a neural network designed to work with sequential data. It uses information from previous time steps along with the current input to make its prediction.
- 2. What information is represented by the hidden state?**
The hidden state stores the important information learned from the previous inputs in a sequence. It acts like a small memory that is carried from one time step to the next.
- 3. Why is sequence order important?**
The order of data can change its meaning, especially in videos, speech and text. For example, changing the order of actions in a video can describe a completely different activity.
- 4. Explain the difference between a feed-forward network and an RNN.**
A feed-forward network processes each input independently without remembering previous inputs. An RNN maintains a hidden state, allowing it to use information from earlier elements of the sequence.
- 5. Explain Backpropagation Through Time.**
Backpropagation Through Time (BPTT) is the method used to train an RNN by unrolling it across its time steps. The error is propagated backward through these time steps to update the network weights.
- 6. Why can gradients vanish during BPTT?**
During BPTT, gradients are repeatedly multiplied across many time steps. If these values become very small, the gradient gradually approaches zero, making it difficult for the network to learn long-term information.
- 7. Why can gradients explode during BPTT?**
If the repeated gradient values become large, their multiplication across many time steps can make the gradient extremely large. This is called the exploding gradient problem and can make training unstable.
- 8. What are long-term dependencies?**
Long-term dependencies occur when the current output depends on information that appeared much earlier in the sequence. For example, an action near the end of a video may depend on something that happened several frames earlier.
- 9. What is the role of the LSTM cell state?**
The LSTM cell state acts as a long-term memory pathway through the sequence. It allows useful information to be carried forward while unwanted information can be removed using gates.
- 10. What are the forget, input and output gates in LSTM?**
The forget gate decides what old information should be removed, while the input gate decides what new information should be stored. The output gate controls which information from the cell state is passed to the next layer.
- 11. What are the reset and update gates in GRU?**
The reset gate controls how much previous information should be ignored when processing new input. The update gate controls how much old information is kept and how much new information is added.
- 12. Compare LSTM and GRU.**
LSTM has a separate cell state and uses three main gates, while GRU combines its memory mechanism into a simpler structure with two gates. GRU generally has fewer parameters and can therefore be faster to train.
- 13. Why does GRU generally use a simpler gating mechanism than LSTM?**
GRU combines some of the functions performed by separate LSTM gates and does not maintain a separate cell state. This gives it a simpler architecture with fewer parameters.
- 14. Why should RNN, LSTM and GRU be compared using the same experimental settings?**
Using the same data, preprocessing, sequence length and evaluation method makes the comparison fair. Otherwise, differences in the experimental setup could affect the results instead of the model architecture itself.
- 15. What does the confusion matrix reveal that accuracy alone does not?**
Accuracy gives only the overall percentage of correct predictions. A confusion matrix shows which particular classes were predicted correctly and which classes were confused with one another.
- 16. Why should macro F1-score be reported for multi-class classification?**
Macro F1-score calculates the F1-score separately for each class and then takes their average. This gives every class equal importance, even when the number of samples in different classes is not the same.
- 17. What is the significance of sequence length?**
Sequence length determines how much temporal information the model receives at one time. A very short sequence may miss important events, while a very long sequence can increase computation and make learning more difficult.
- Why can a CNN be used as a feature extractor for video?**
A CNN is good at identifying useful visual patterns in individual images. Since a video is made of frames, a CNN can extract spatial features from each frame and these features can later be processed as a sequence.

19. **What type of information is extracted by the CNN?**

The CNN mainly extracts spatial information from each frame, such as objects, shapes, textures, appearance and body positions. It describes what is visible in an individual frame.

20. **What type of information is learned by the recurrent network?**

The recurrent network learns temporal information by looking at the sequence of CNN feature vectors. It learns how the visual information changes from one frame to another over time.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

Each frame produces one CNN feature vector, but a video contains multiple frames in a particular order. Arranging these vectors as a sequence allows the LSTM or GRU to learn the temporal relationships between the frames.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

The encoder reads the input sequence and converts it into a learned representation. The decoder uses this representation to generate the required output sequence step by step.

23. **What is teacher forcing?**

Teacher forcing is a training technique where the decoder receives the correct previous output instead of its own previous prediction. This helps the decoder learn the target sequence more quickly during training.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy checks how many individual output elements are predicted correctly. Sequence accuracy is stricter because the entire sequence must be correct for it to count as a correct prediction.

25. **Why must the test set remain untouched during model selection?**

The test set should represent completely unseen data. If it is used while choosing or tuning the model, information from the test set can influence the decisions and give an overly optimistic estimate of final performance.

28. Additional Exercises

1. **Effect of the Number of Recurrent Units**

The original 32-unit RNN was compared with 16 and 64 units. The same 128-step, 9-feature HAR sequences were used for all models.

Table 1: Effect of recurrent units on RNN performance

Units	Accuracy	F1-score	Parameters	Training Time (s)
16	0.5453	0.4965	518	31.57
32	0.6790	0.6588	1,542	29.00
64	0.4428	0.4209	5,126	31.39

Inference: Increasing the units from 16 to 32 improved the results, but 64 units performed worse in this run. This shows that simply adding more recurrent units does not always improve performance.

2. **GRU vs LSTM**

Both models used 32 recurrent units and the same training settings.

Table 2: Comparison of GRU and LSTM

Model	Accuracy	F1-score	Parameters	Training Time (s)
LSTM	0.6865	0.6796	5,574	50.62
GRU	0.8459	0.8420	4,326	79.12

Inference: In this experiment, the GRU achieved higher accuracy and F1-score with fewer parameters than the LSTM. The measured training time was higher for the GRU in this particular run.

3. **Adding a Second Recurrent Layer**

A single 32-unit LSTM was compared with a two-layer LSTM containing 32 units in each layer.

Table 3: Effect of adding a second LSTM layer

Model	Accuracy	F1-score	Parameters	Training Time (s)
Single-Layer LSTM	0.7377	0.7274	5,574	63.21
Two-Layer LSTM	0.8734	0.8735	13,894	101.50

Inference: Adding the second LSTM layer substantially improved both accuracy and F1-score in this run. The improvement came with more parameters and longer training time.

4. Bidirectional LSTM vs Unidirectional LSTM

Both models used 32 LSTM units.

Table 4: Unidirectional and bidirectional LSTM comparison

Model	Accuracy	F1-score	Parameters	Training Time (s)
Unidirectional LSTM	0.8178	0.8131	5,574	50.24
Bidirectional LSTM	0.8738	0.8731	11,142	89.88

Inference: The bidirectional LSTM produced higher accuracy and F1-score than the unidirectional model. It also used about twice the parameters and required more training time.

5. Effect of Sequence Length

The LSTM was evaluated using sequence lengths of 32, 64, 96, and 128 while keeping the recurrent units fixed at 32.

Table 5: Effect of sequence length on performance and computational cost

Sequence Length	Accuracy	F1-score	Parameters	Training Time (s)
32	0.8266	0.8229	5,574	17.41
64	0.8548	0.8522	5,574	32.99
96	0.8565	0.8545	5,574	43.97
128	0.8303	0.8264	5,574	59.58

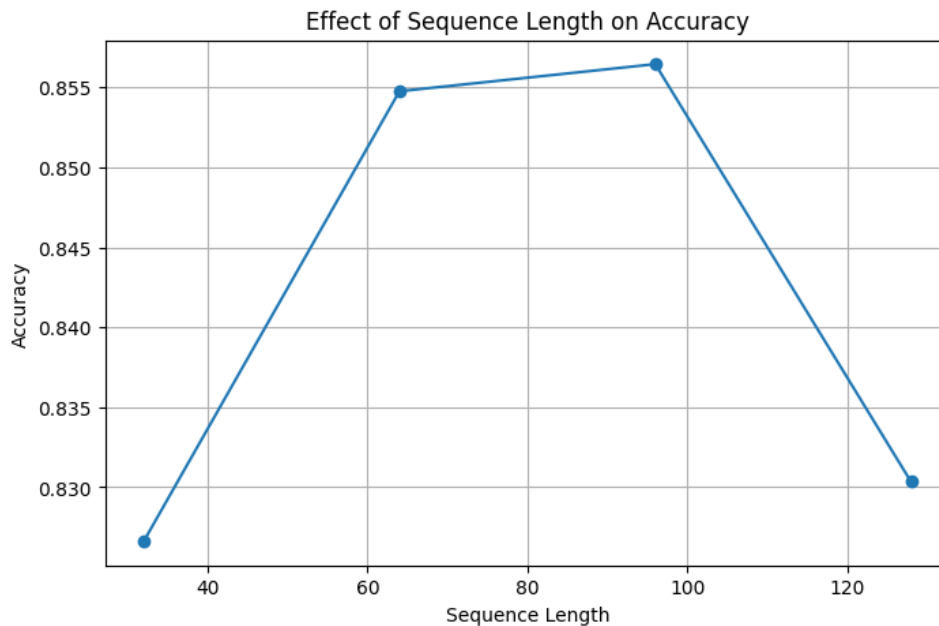


Figure 18: Effect of sequence length on training time.

Inference: Training time increased steadily as the sequence became longer, while the best accuracy in this run was obtained at length 96. This suggests that longer sequences increase computational cost without guaranteeing better classification performance.

6. Video Task: LSTM vs GRU with Identical CNN Features

This exercise was specified in the notebook task, but no completed video LSTM/GRU result is recorded in the uploaded notebook. Therefore, no numerical result is added here.

7. Seq2Seq with Different Input and Output Lengths

This exercise was specified in the notebook task, but no completed seq2seq result is recorded in the uploaded notebook. Therefore, no numerical result is added here.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

31. Resources

- Google Colab Notebook₁ :
Google Colab Notebook₂ :
Google Colab Notebook₃(*AdditionalTask*) :
GitHub Repository: https://github.com/LaasyaPriya-1238/DL_LAB